Doc. No.:	WG21/P0668R4
Date:	2018-06-24
Reply-to:	Hans-J. Boehm
Email:	hboehm@google.com
Authors:	Hans-J. Boehm, Olivier Giroux, Viktor Vafeiades, with input from from Will Deacon, Doug Lea, Daniel Lustig, Paul McKenney and others
Audience:	CWG, LWG
Note to editor:	***Consolidated wording is in the last section***

P0668R4: Revising the C++ memory model

Although the current C++ memory model, adopted essentially in C++11, has served our user community reasonably well in practice, a number of problems have come to light. The first one of these is particularly new and troubling:

1. Existing implementation schemes on Power and ARM are not correct with respect to the current memory model definition. These implementation schemes can lead to results that are disallowed by the current memory model when the user combines acquire/release ordering with seq_cst ordering. On some architectures, especially Power and Nvidia GPUs, it is expensive to repair the implementations to satisfy the existing memory model. Details are discussed in (Lahav et al) http://plv.mpi-sws.org/scfix/paper.pdf (which this discussion relies on heavily). The same issues were briefly outlined at the Kona SG1 meeting. We summarize below.
2. Our current definition of `memory_order_seq_cst`, especially for fences, is too weak. This was caused by historical assumptions that have since been disproved.
3. The current definition of *release sequence* is problematic, allowing seemingly irrelevant `memory_order_relaxed` operations to interfere with synchronizes-with relationships.
4. We still do not have an acceptable way to make our informal (since C++14) prohibition of out-of-thin-air results precise. The primary practical effect of that is that formal verification of C++ programs using relaxed atomics remains unfeasible. The above paper suggests a solution similar to http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3710.html . We continue to ignore the problem here, but try to stay out of the way of such a solution.
5. The current definition of `memory_order_consume` is widely acknowledged to be unusable, and implementations generally treat it as `memory_order_acquire`. The current draft "temporarily discourages" it. See http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0371r1.html . There are proposals to repair it (cf. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0190r3.pdf and http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0462r1.pdf), but nothing that is ready to go.

Here we concentrate on, and outline proposals for, the first two. The third one is now the subject of a separate proposal, P0982, since it seemed to generate more of an SG1 consensus than the others in Jacksonville. The last two are not addressed here, but should be kept in mind. In our view, the first of these is currently the most pressing.

Power, ARM, and GPU implementability

Although we previously believed otherwise, it has recently been shown that the standard implementations of memory_order_acquire and memory_order_release on Power are insufficient. Very briefly, these are compiled using "lightweight" fences, which are insufficient to enforce required properties for memory_order_sq_cst accesses to the same location.

To illustrate, we borrow the Z6.U example, from S 2.1 of http://plv.mpi-sws.org/scfix/paper.pdf (pseudo-C++ syntax, memory orders indicated by subscript, e.g. $y =_{rel} 1$ abbreviates `y.store(memory_order_release, 1)`):

Thread 1	Thread 2	Thread 3
$x =_{sc} 1$	$b = fetch\_add(y)_{sc}$ //1	$y =_{sc} 3$
$y =_{rel} 1$	$c = y_{rlx}$ //3	$a = x_{sc}$ //0

The comments here indicate the observed assigned value.

The indicated outcome here is disallowed by the current standard: All `memory_order_seq_cst` (sc) accesses must occur in a single total order, which is constrained to have **$a = x_{sc}$ //0** before **$x =_{sc} 1$** (since it doesn't observe the store), which must be before **$b = fetch\_add(y)_{sc}$ //1** (since it happens before it), which must be before **$y =_{sc} 3$** (since the fetch_add does not observe the store, which is forced to be last in modification order by the load in Thread 2). But this is disallowed since the standard requires the happens before ordering to be consistent with the sequential consistency ordering, and **$y =_{sc} 3$**, the last element of the sc order, happens before **$a = x_{sc}$ //0**, the first one.

On the other hand, this outcome is allowed by the Power implementation. Power normally uses the "leading fence" convention for sequentially consistent atomics. (See http://www.cl.cam.ac.uk/~pes20/cpp/cpp0xmappings.html) This means that there is only an lwsync fence between Thread 1's instructions. This does not have the "cumulativity"/transitivity properties that would be required to make the store to `x` visible to Thread 3 under these circumstances.

This issue was missed in earlier analyses. This example is not a problem for the "trailing fence" mapping that could also have been used on Power. But Lahav et al contains examples that fail for that mapping as well, for similar reasons.

This example relies crucially on the fact that a `memory_order_release` operation synchronizes with a `memory_order_seq_cst` operation on the same location. Code that consistently accesses each location only with `memory_order_seq_cst`, or only with weaker ordering, is not affected, and works correctly.

Whether or not such code occurs in practice depends on coding style. One reasonable coding style is to initially use only seq_cst operations, and then selectively weaken those that are performance critical; it does result in such cases. Even in such cases, it seems clear that the current compilation strategy does not result in frequent failures; this problem was discovered through careful theoretical analysis, not bug reports. It is unclear whether there is any real code that can fail as a result of the current mapping; it would require careful analysis of the use cases to determine whether the weaker ordering provided by the hardware is in fact sufficient for these use cases.

For ARM, the situation is theoretically similar, but appears to be much less severe in practice. On ARMv8, the usual compilation mode for loads and stores is currently to compile acquire/release operations as seq_cst operations, so there is currently no issue. On ARMv7, some compilation schemes for acquire/release have the same issues as for Power, but the most common scheme seems to be to use "dmb ish", which does not share this problem.

Nvidia GPUs have a memory model similar to Power, and share the same issues, probably with larger cost differences. For very large numbers of cores, it is natural to share store buffers between threads, which may make stores visible to different threads in inconsistent orders. This lack of "multi-copy atomicity" is also the core distinguishing property of the Power and ARM memory models. We suspect that other GPUs are also affected, but cannot say that definitively.

We are not aware of issues on other existing CPU architectures. Since it appears more attractive to drop multi-copy atomicity with higher core counts, we expect the same issue to recur with some future processor designs.

Alternative 1: "Just" fix the implementations

Repairing this on Power without changing the specification would prevent us from generating the lighter weight "lwsync" fence instruction for a `memory_order_release` operation (unless we either know it will never synchronize with a `memory_order_seq_cst` operation, or we make `memory_order_seq_cst` operations even more expensive), which would have a significant performance impact on acquire/release synchronization. It would also defeat a significant part, though not all, of the motivation for for introducing `memory_order_acquire` and `memory_order_release` to start with.

The cost on GPUs is likely to be higher.

Among people we informally surveyed, this was not a popular option. Many people felt that we would be penalizing a subset of the available machine architectures for an issue with little practical impact. The language would no longer be able to express pure acquire-release synchronization, which many people feel is essential.

We could regain acquire-release synchronization by adding a new "weak" atomic type that does not support `memory_order_seq_cst`, and requires explicit `memory_order` arguments. Again there was general concern that this significantly increases the library API footprint for an issue without much practical impact.

Alternative 2: Fix the standard

This is the approach taken by Lahav et al, and the one we pursue here.

The proposal in Lahav et al is mathematically elegant. Currently the standard requires that the sequential consistency total order S is consistent with the happens before relation. Essentially, if any two sc operations are ordered by happens before, then they must be ordered the same way by S. In our example, this requires $x =_{sc} 1$ to be ordered before $b = \text{fetch\_add}(y)_{sc}$ //1 in spite of the fact that the hardware mapping does not sufficiently enforce it. The core fix (S1fix in the paper) is to relax the restriction that a happens before ordering implies ordering in S to only the sequenced before case, or the case in which the happens before ordering between *a* and *b* is produced by a chain

a is sequenced before *x* happens before *y* is sequenced before *b*

The downside of this is that "happens before" now has a rather strange meaning, since sequentially consistent operations can appear to execute in an order that's not consistent with it.

In the Z6.U example, **`x =`$_{sc}$ `1`** must no longer precede **`b = fetch_add(y)`$_{sc}$ `//1`** in the sequential consistency order S, in spite of the fact that the former "happens before" the latter. And in the questionable execution that we now wish to allow, they indeed have the opposite order in S.

We propose to make this somewhat less confusing by suitably renaming the relations in the standard as follows:

Currently the initialization rules, etc. use "strongly happens before" in guaranteeing ordering. The current intent is to also use that to specify library ordering, such as for mutexes. We propose to modify that definition to require "sequenced before" ordering at both ends. This new improved "strongly happens before" would be used in the same contexts as now, and would remain strong enough to ensure that if *a* happens before *b*, and they both participate in the sc ordering S, then *a* also precedes *b* in S. "Strongly happens before" would continue to exclude any ordering via memory_order_consume, since such ordering is much more restrictive, and must be explicitly accommodated by the caller.

Thus we would propose to change 6.8.2.1p11 [intro.races, once known as 1.10] as follows:

> An evaluation A ~~*strongly*~~ *simply* *happens before* an evaluation B if either
>
> - A is sequenced before B, or
> - A synchronizes with B, or
> - A simply happens before X and X simply happens before B.
>
> [*Note:* In the absence of consume operations, the happens before and ~~strongly~~ simply happens before relations are identical. ~~Strongly happens before essentially excludes consume operations.~~ — *end note*]
>
> An evaluation A *strongly happens before* an evaluation D if, either
>
> - A is sequenced before D, or
> - A synchronizes with D, and both A and D are sequentially consistent atomic operations (32.4 [atomics.order]), or
> - there are evaluations B and C such that A is sequenced before B, B simply happens before C, and C is sequenced before D, or
> - there is an evaluation B such that A strongly happens before B, and B strongly happens before D.
>
> [*Note:* Informally, if A strongly happens before B, then A appears to be evaluated before B in all contexts. Strongly happens before excludes consume operations.--*end note*]

We would then adjust 32.4p3 [atomics.order] correspondingly:

> There shall be a single total order S on all `memory_order_seq_cst` operations, consistent with the "strongly happens before" order, and modification orders for all affected locations, such that each

> `memory_order_seq_cst` operation B that loads a value from an atomic object M observes one of the following values: …

and add a second note at the end of p3:

> [*Note:* We do not require that S be consistent with "happens before" (6.8.2.1 [intro.races]). This allows more efficient implementation of `memory_order_acquire` and `memory_order_release` on some machine architectures. It may produce more surprising results when these are mixed with `memory_order_seq_cst` accesses. -- *end note*]

Note for editor: Please use the consolidated wording at the end instead to resolve conflicts between sections.

Terminology and teachability

There was a fair amount of discussion in Jacksonville about the number of different "happens before" variants, whether we plain "happens before" is correctly used for the most important variant, and whether this is all understandable. While we share some of this concern, we believe that the choices presented above represent the best possible option.

We are not touching the requirement that user code avoid data races. In the absence of weakly ordered atomics, this simply requires that conflicting operations not be executed concurrently. And programmers need not concern themselves with any of the happens-before variants. (We've added a few additional constructs, beyond weakly ordered atomics, that violate this. But so far such visible violations have been limited to code that is clearly unreasonable for other reasons.)

Once we throw weakly-ordered atomics into the mix, the major concern that programmers should have is whether this relaxation introduces data races. That means that conflicting operations must be ordered by happens-before. That also remains unchanged. The plain happens-before relation is unchanged, and continues to be the right one to use here.

The definition of plain happens-before became unpleasantly complicated with the introduction of `memory_order_consume`. And it is not transitive, which remains counterintuitive. This proposal changes neither of those. And if the user refrains from using `memory_order_consume` it can continue to be entirely ignored, as before. Until we have a usable version of `memory_order_consume`, I would expect teaching materials to ignore these issues, and pretend that happens-befors is defined as our simply-happens-before relation, which is clearly transitive. In the presence of `memory_order_consume`, this problem is unavoidable, since consume can order two accesses without also ordering the first with respect to an access that immediately follows the second; happens-before cannot compose with sequenced-before, and thus happens-before cannot be transitive.

Unfortunately, the plain happens-before relation is not the right one to use for most library writers. It does not suffice for a library to promise that a call A happens-before a call B, since the library user will generally also need to conclude that if B is immediately followed by another call C, then A also happens before C. And we want to make sure that happens-before promises also translate into guarantees about the ordering of sequentially-consistent operations. By default libraries should hide their internal weaknesses, and ensure correct usablility in all contexts. Thus they must ensure a *stronger* notion of happens-before that guarantees proper composability in all contexts. That is the purpose of strong-happens-before.

The fundamental role of strong-happens-before also is not changed by this proposal. However its definition has changed, since the old definition would no longer imply consistency with the SC ordering, and would thus allow surprising results in a few esoteric cases.

The definition of strong-happens-before now effectively requires that

- `memory_order_consume` is not used to establish happens-before, and
- Any use of synchronizes-with in establishing the ordering must either use SC operations at both ends, or preceded and followed by a sequence-based intra-thread ordering.

This definition is made appreciably more complicated by this proposal, but in a way that should not concern most library writers either. The only case it newly precludes is the one in which the second constraint is violated. Normally the strongly-happens-before guarantee will be used to order the users code before and after the calls, meaning that the requirement is implicitly satisfied by the addition of user code. It only matters when the library calls themselves are promised to participate in the SC ordering, so that the library user will use the happens-before relation to infer properties of the SC ordering. And even in that case, the implementation is likely to use SC operations at both ends, again avoiding any issues. We do not know of a way to simplify this while preserving correctness with respect to current widespread implementation strategies.

Summarizing, we continue to have, as we did before:

(plain) happens-before
: The fundamental relation used to reason about data races

strong happens before
: The ordering relation usually promised by libraries to ensure usability and composability in all contexts

The simply-happens-before relation is an atifact of our definition. In the absence of

1. `memory_order_consume`, and
2. mixed use of both acquire/release and seq_cst operations on the same location

all three definitions coincide.

Strengthen SC fences

The current `memory_order_seq_cst` fence semantics do not guarantee that a program with only memory_order_relaxed accesses and `memory_order_seq_cst` fences between each pair actually exhibits sequential consistency. This was, at one point, intentional. The goal was to ensure that architectures like Itanium that allow stores to become visible to different processors in different orders, and do not provide fences to rectify this, could be supported. But it subsequently became clear that Itanium, as a result of failing to provide strong ordering for accesses to a single location) would need to use stronger primitives for memory_order_relaxed anyway. All known SC fence implementations provide the stronger semantics, and we should acknowledge that.

We propose to strengthen the `memory_order_seq_cst` fence semantics as suggested in Lahav et al. (Note edit conflict with last section; please use the consolidated wording below to resolve.)

Replace Section 32.4 [atomics.order] paragraphs 3-7 (the definition of SC ordering) with:

An atomic operation A on some atomic object M is *coherence-ordered before* another atomic operation B on M if

- A is a modification, and B observes A, or
- A precedes B in the modification order, or
- A and B are not the same atomic read-modify-write operation, and there exists an atomic modification X of M such that A observes the value written by X and X precedes B in modification order, or
- there exists X such that A is coherence-ordered before X and X is coherence-ordered before B.

There shall be a single total order S on all `memory_order_seq_cst` operations, consistent with the "strongly happens before" order, such that for every pair of atomic operations A and B on an object M, where A is coherence-ordered before B,

- if A and B are both `memory_order_seq_cst` operations, then A precedes B in S; and
- if A is a `memory_order_seq_cst` operation and B happens before a `memory_order_seq_cst` fence Y, then A precedes Y in S; and
- if a `memory_order_seq_cst` fence X happens before A and B is a `memory_order_seq_cst` operation, then X precedes B in S; and
- if a `memory_order_seq_cst` fence X happens before A and B happens before a `memory_order_seq_cst` fence Y, then X precedes Y in S.

[*Note:* This definition ensures that S is consistent with the modification order. It also ensures that a `memory_order_seq_cst` read of an atomic object M gets its value either from the last modification of M that precedes A in S or from some non-`memory_order_seq_cst` modification of M that does not happen before any modification of M that precedes A in S. -- *end note*]

The note from the previous section would go after this.

Finally adjust the existing note in the immediately following p8 as follows, to remove a statement that is no longer correct:

[*Note:* `memory_order::seq_cst` ensures sequential consistency only for a program that is free of data races and uses exclusively `memory_order::seq_cst` atomic operations. Any use of weaker ordering will invalidate this guarantee unless extreme care is used. In many cases, `memory_order_seq_cst` atomic operations may be reordered with respect to other atomic operations performed by the same thread. ~~In particular, `memory_order::seq_cst` fences ensure a total order only for the fences themselves. Fences cannot, in general, restore sequential consistency for atomic operations with weaker ordering specifications.~~ — *end note*]

The definition of "coherence-ordered before" is essentially standard terminology, but was not previously part of our standard. The third bullet corresponds to what's often called "reads before": A reads a write earlier than B.

This new wording takes a significantly different approach with respect to the sequential consistency ordering S: Instead of defining visibility in terms of S and the other orders in the standard, this essentially defines constraints on S in terms of visibility in a particular execution, as expressed by the coherence order. If these constraints are

not satisfiable by any total order S, then the candidate execution which gave rise to the coherence order is not valid.

History

P0668R4

Reflected Rapperswil CWG comments in the wording. At the request of CWG and the editor, added consolidated wording section below.

Added a fix for the note in 32.4p8 that was previously overlooked.

P0668R3

In response to SG1 discussion in Jacksonville:

- Separated out the release sequence modifications into a separate paper, P0982.
- Added a section to justify the naming of happens-before variants and explain the intended model for teaching. SG1 was concerned that this would be hard to explain to a wider audience. This concern was also previously raised in discussions among the authors.

The first clause in the definition of "coherence ordered before" was stated backwards. Fixed.

P0668R2

Added wording for weakening release sequence guarantee. Adjust section numbering to N4713.

P0668R1

In Toronto, we discussed an update D0668R1 of P0668R0 that added wording for the sequentially consistent fence changes, and that added the release sequence proposal. The first two proposals received strong support for the core idea; it was understood that the precise wording needed more bake time.

The vote on the release sequence proposal was delayed after hardware architects pointed out that it potentially imposed significant hardware constraints. It made sense to reexamine the significance of the underlying problem to make sure that the change was justified, particularly since the entire argument for allowing same thread stores to extend a release sequence now seems suspect.

This version incorporates the changes from the draft document we discussed, fixes some serious editing mistakes in D0668R1, adds further discussion for the release sequence proposal, and adjusts the desired straw poll list.

P0668R0

Initial version.

Consolidated wording:

This is relative to N4750.

Change 6.8.2.1p11 [intro.races] as follows:

> An evaluation A ~~*strongly*~~ *simply* *happens before* an evaluation B if either
>
> - A is sequenced before B, or
> - A synchronizes with B, or
> - A simply happens before X and X simply happens before B.
>
> [*Note:* In the absence of consume operations, the happens before and ~~strongly~~ simply happens before relations are identical. ~~Strongly happens before essentially excludes consume operations.~~ — *end note*]
>
> An evaluation A *strongly happens before* an evaluation D if, either
>
> - A is sequenced before D, or
> - A synchronizes with D, and both A and D are sequentially consistent atomic operations (32.4 [atomics.order]), or
> - there are evaluations B and C such that A is sequenced before B, B simply happens before C, and C is sequenced before D, or
> - there is an evaluation B such that A strongly happens before B, and B strongly happens before D.
>
> [*Note:* Informally, if A strongly happens before B, then A appears to be evaluated before B in all contexts. Strongly happens before excludes consume operations.--*end note*]

Replace Section 32.4 [atomics.order] paragraphs 3-7, which currently reads:

> ~~There shall be a single total order S on all memory_order::seq_cst operations, consistent with the "happens before" order and modification orders for all affected locations, such that each `memory_order::seq_cst` operation B that loads a value from an atomic object M observes one of the following values:~~
>
> 1. ~~the result of the last modification A of M that precedes B in S, if it exists, or~~
> 2. ~~if A exists, the result of some modification of M that is not `memory_order::seq_cst` and that does not happen before A, or~~
> 3. ~~if A does not exist, the result of some modification of M that is not `memory_order::seq_cst`.~~
>
> ~~[*Note:* Although it is not explicitly required that S include locks, it can always be extended to an order that does include lock and unlock operations, since the ordering between those is already included in the "happens before" ordering. — *end note*]~~
>
> ~~For an atomic operation B that reads the value of an atomic object M, if there is a `memory_order::seq_cst` fence X sequenced before B, then B observes either the last `memory_order::seq_cst` modification of M preceding X in the total order S or a later modification of M in its modification order.~~
>
> ~~For atomic operations A and B on an atomic object M, where A modifies M and B takes its value, if there is a `memory_order::seq_cst` fence X such that A is sequenced before X and B follows X in S,~~

~~then B observes either the effects of A or a later modification of M in its modification order.~~

~~For atomic operations A and B on an atomic object M, where A modifies M and B takes its value, if there are `memory_order::seq_cst` fences X and Y such that A is sequenced before X, Y is sequenced before B, and X precedes Y in S, then B observes either the effects of A or a later modification of M in its modification order.~~

~~For atomic modifications A and B of an atomic object M, B occurs later than A in the modification order of M if:~~

1. ~~there is a `memory_order::seq_cst` fence X such that A is sequenced before X, and X precedes B in S, or~~
2. ~~there is a `memory_order::seq_cst` fence Y such that Y is sequenced before B, and A precedes Y in S, or~~
3. ~~there are `memory_order::seq_cst` fences X and Y such that A is sequenced before X, Y is sequenced before B, and X precedes Y in S.~~

with:

An atomic operation A on some atomic object M is *coherence-ordered before* another atomic operation B on M if

- A is a modification, and B observes A, or
- A precedes B in the modification order, or
- A and B are not the same atomic read-modify-write operation, and there exists an atomic modification X of M such that A observes the value written by X and X precedes B in modification order, or
- there exists X such that A is coherence-ordered before X and X is coherence-ordered before B.

There shall be a single total order S on all `memory_order_seq_cst` operations, consistent with the "strongly happens before" order, such that for every pair of atomic operations A and B on an object M, where A is coherence-ordered before B,

- if A and B are both `memory_order_seq_cst` operations, then A precedes B in S; and
- if A is a `memory_order_seq_cst` operation and B happens before a `memory_order_seq_cst` fence Y, then A precedes Y in S; and
- if a `memory_order_seq_cst` fence X happens before A and B is a `memory_order_seq_cst` operation, then X precedes B in S; and
- if a `memory_order_seq_cst` fence X happens before A and B happens before a `memory_order_seq_cst` fence Y, then X precedes Y in S.

[*Note:* This definition ensures that S is consistent with the modification order. It also ensures that a `memory_order_seq_cst` read of an atomic object M gets its value either from the last modification of M that precedes A in S or from some non-`memory_order_seq_cst` modification of M that does not happen before any modification of M that precedes A in S. -- *end note*]

[*Note:* We do not require that S be consistent with "happens before" (6.8.2.1 [intro.races]). This allows more efficient implementation of `memory_order_acquire` and `memory_order_release` on some machine architectures. It may produce more surprising results when these are mixed with `memory_order_seq_cst` accesses. -- *end note*]

Finally adjust the existing note in the immediately following p8 as follows, to remove a statement that is no longer correct:

[*Note:* `memory_order::seq_cst` ensures sequential consistency only for a program that is free of data races and uses exclusively `memory_order::seq_cst` atomic operations. Any use of weaker ordering will invalidate this guarantee unless extreme care is used. In many cases, `memory_order_seq_cst` atomic operations may be reordered with respect to other atomic operations performed by the same thread. ~~In particular, `memory_order::seq_cst` fences ensure a total order only for the fences themselves. Fences cannot, in general, restore sequential consistency for atomic operations with weaker ordering specifications.~~ — *end note*]